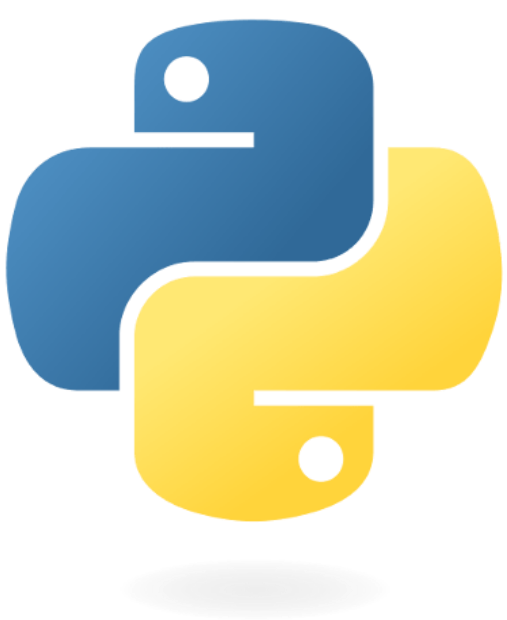


python™

Classroom code
ut9ntoa

Prof. Luciano Nunes
Jun/2018
lnunes at gmail

Algoritmos e Programação em Linguagem Python

Aula1

- Linguagens de Programação
- Importância da Orientação a Objetos
- Boas práticas de Programação
- Algoritmo
- Ambiente de Desenvolvimento
- Introdução ao Python
  - Sintaxe Básica
  - Variáveis
  - Tipos de Dados
  - Funções mais comuns
  - Controle de Fluxo
  - Funções
  - Operadores

Aula2

- Módulos e Pacotes
  - datetime
  - time
  - calendar
- Entrada e Saída (I/O)
- Erros e Exceções
  - Captura e Tratamento de Exceções

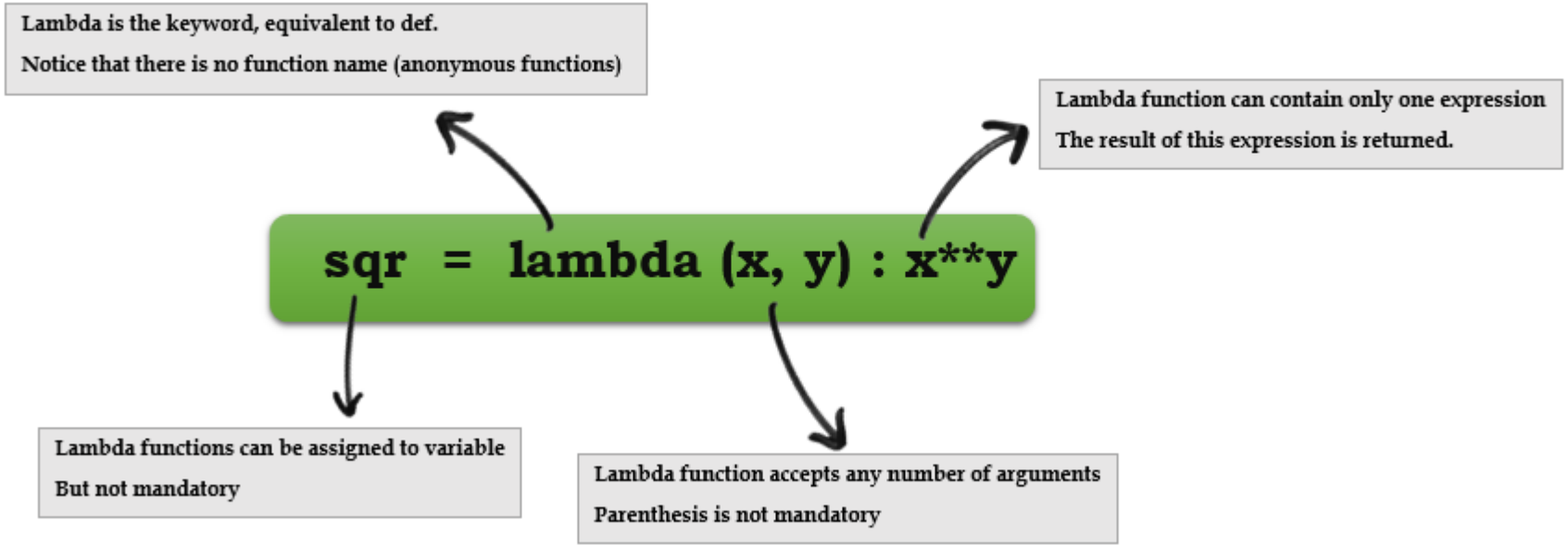
Aula 3 - Final do módulo

- Lambda, map, filter e zip
- List Comprehension
- Generator
- Generator Expression
- Orientação a Objetos
- Multithreading



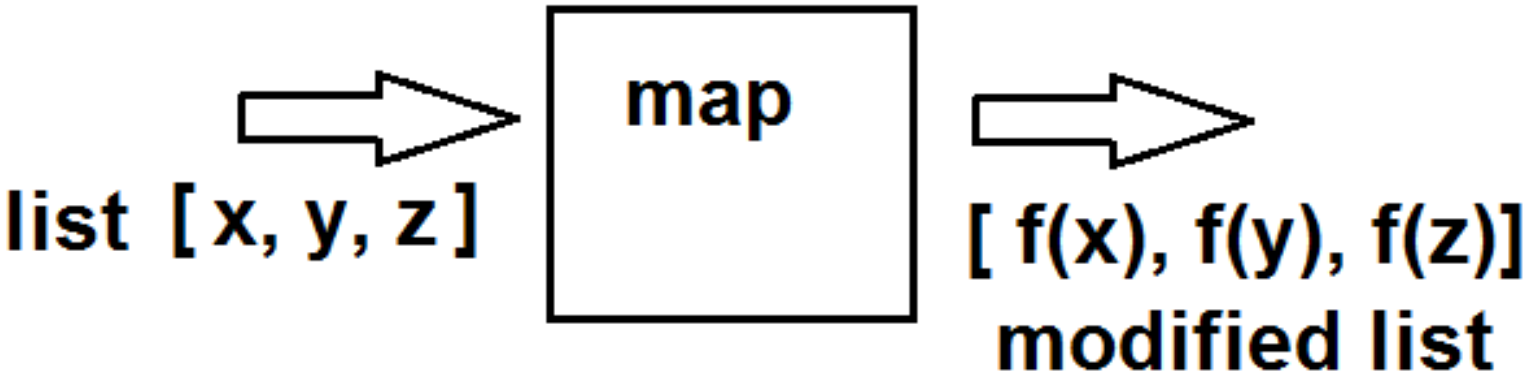
Lambda

- Conceito e nomes vieram da linguagem funcional Lisp
- São funções anônimas que aceitam argumentos e que só suportam uma expressão
- São usadas basicamente quando há a necessidade de uma função que não seja complexa o suficiente que justifique sua criação
- Nesse tipo de expressão, o Python retorna a própria função, invés de atribuí-la a um nome como acontece em um statement "def", daí vem a particularidade de serem anônimas



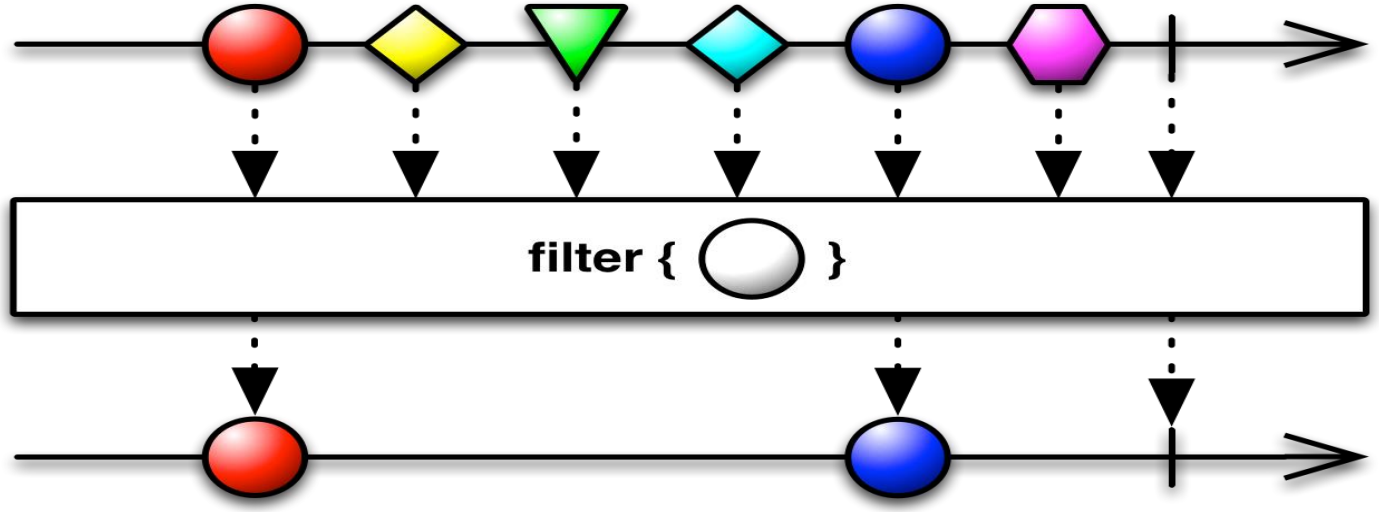
Map

- Map é uma função embutida (*built in*) do Python
- Basicamente map executa uma determinada função a uma sequência de valores, aplicando essa função a cada elemento dessa sequência
- Utilizado em conjunto com as instruções lamba, podem ser muito úteis para a transformação de elementos de uma lista de forma massiva



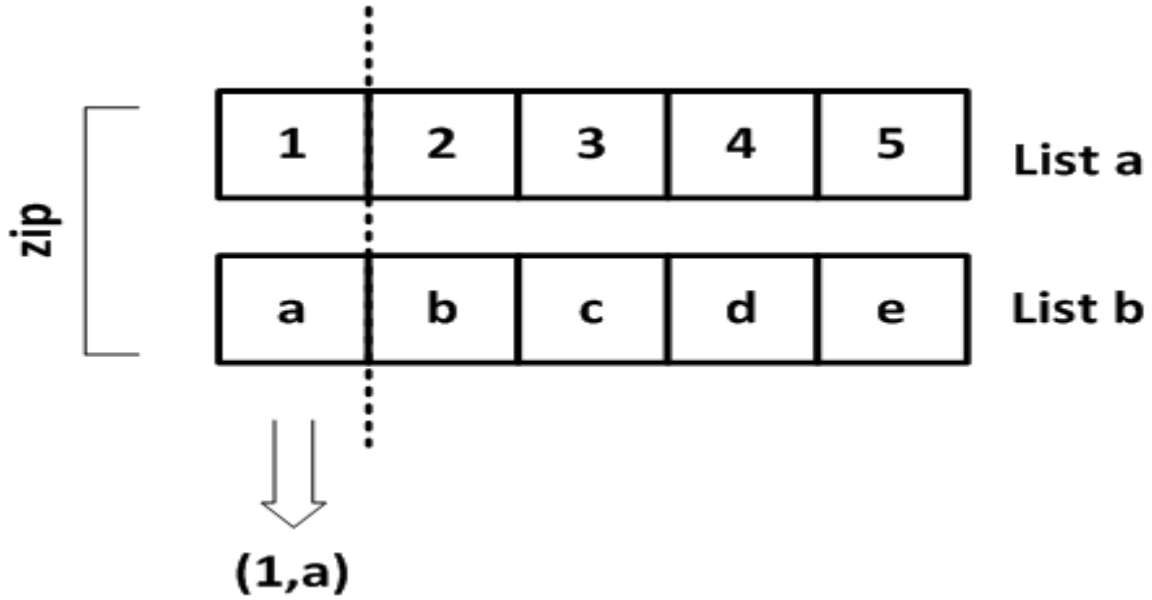
Filter

- Filter é uma função embutida (*built in*) do Python
- Irá iterar e retornar um iterador para os elementos de uma lista, caso a condição da função passada como primeiro argumento seja satisfatória
- Admite None como parâmetro da função, irá retornar um iterador para os elementos verdadeiros da lista de entrada
- Assim como o map, pode ser utilizado em conjunto com instruções lambda



Zip

- Zip é uma função embutida (*built in*) do Python
- Retorna um iterador para uma lista com a combinação dos elementos de cada iterador passado como argumento
- Caso os iteradores passados como argumentos tenham tamanhos diferentes, será retornado um iterador com o tamanho do menor iterador passado como argumento



List Comprehension

- Embora as expressões lambda e suas associações com funções built in do Python como map e filter sejam interessantes, o criador do Python optou por mantê-las na linguagem apenas para atrair programadores de outras linguagens funcionais como Lisp
- No entanto sua preferência é pelas *list comprehensions* que nada mais são do que a forma *Pythonica* ou mais elegante de escrever a mesma coisa
- Portanto, praticamente tudo o que é possível de se fazer usando expressões lambda, map e filter, talmbém é possivel usando *list comprehensions*
- Tanto em desempenho quanto em concisão ambas as formas são bastante semelhantes, apenas o estilo de programação é diferente

```
[expression for name in list if filter]
```

```
>>> li = [1,2,3,4,5,6,7,8,9,10]
>>> [x**2 for x in li if x%2==0]
```

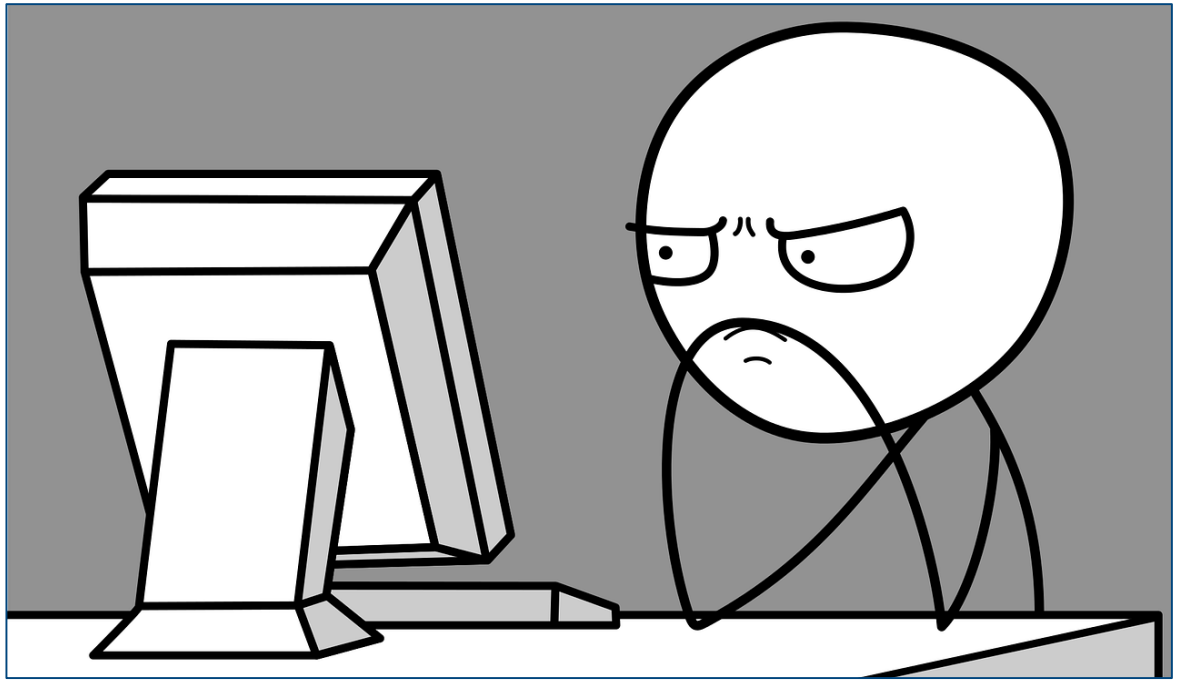
```
[4, 16, 36, 64, 100]
```

Lambdas X List Comprehension

comprehensions.py

```
1   # imperative form
2   odd = []
3   for i in range(1000):
4       if i % 2:
5           odd.append(i)
6   print(odd)
7
8   # functional form
9   print(list(filter(lambda i: i % 2, range(1000))))
10
11  # comprehension form
12  print([i for i in range(1000) if i % 2])
13
14
15
```

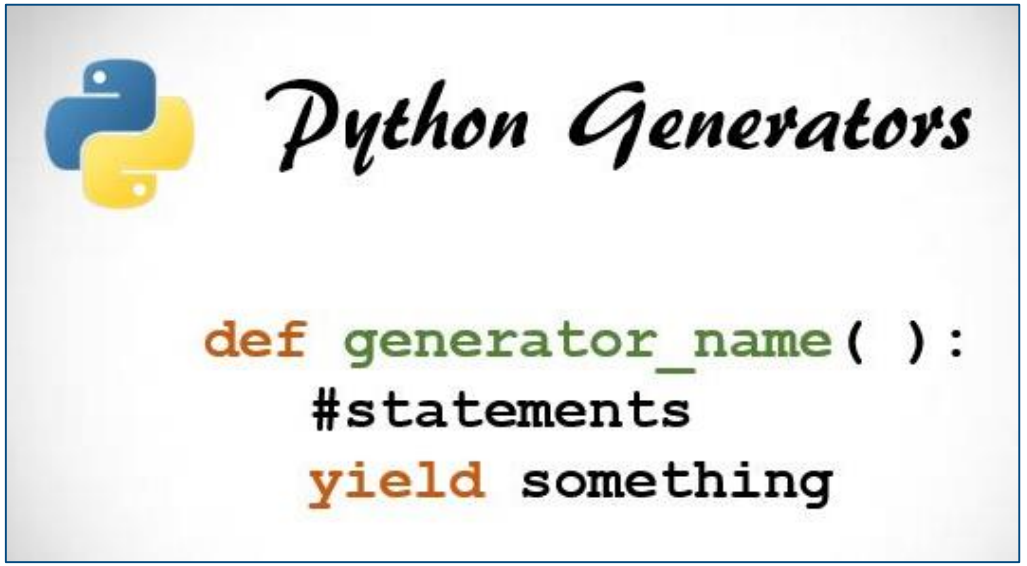
Lambdas X List Comprehension



- Umas das questões muito discutidas é sobre a complexidade que pode estar relacionada à utilização de expressões lambdas complexas, assim como list comprehensions com muitas camadas de lógica aninhadas
- Procure manter um único estilo
- Eventualmente várias linhas de código podem ser subistituídas por apenas uma linha utilizando-se desse tipo de facilidade que a linguagem oferece, no entando é importante utilizar esses recursos com sabedoria, lembrando-se sempre de comentar seu código
- Você pode imaginar-se não compreendendo um programa escrito por você mesmo 6 meses atrás?

Generators

- Generators são uma forma que a linguagem Python oferece para a escrita de funções que tem o comportamento de um iterador
- Simplificando é uma forma de criarmos nossos próprios iteradores
- A principal particularidade de um generator é sua capacidade de preservar os valores das variáveis locais
- Para que isso seja possível, o *statement* ***return*** deve ser substituído por ***yield***
- Ao encontrar essa instrução, o interpretador devolve o controle para a função chamadora, preservando os valores das variáveis locais até a próxima chamada ou iteração



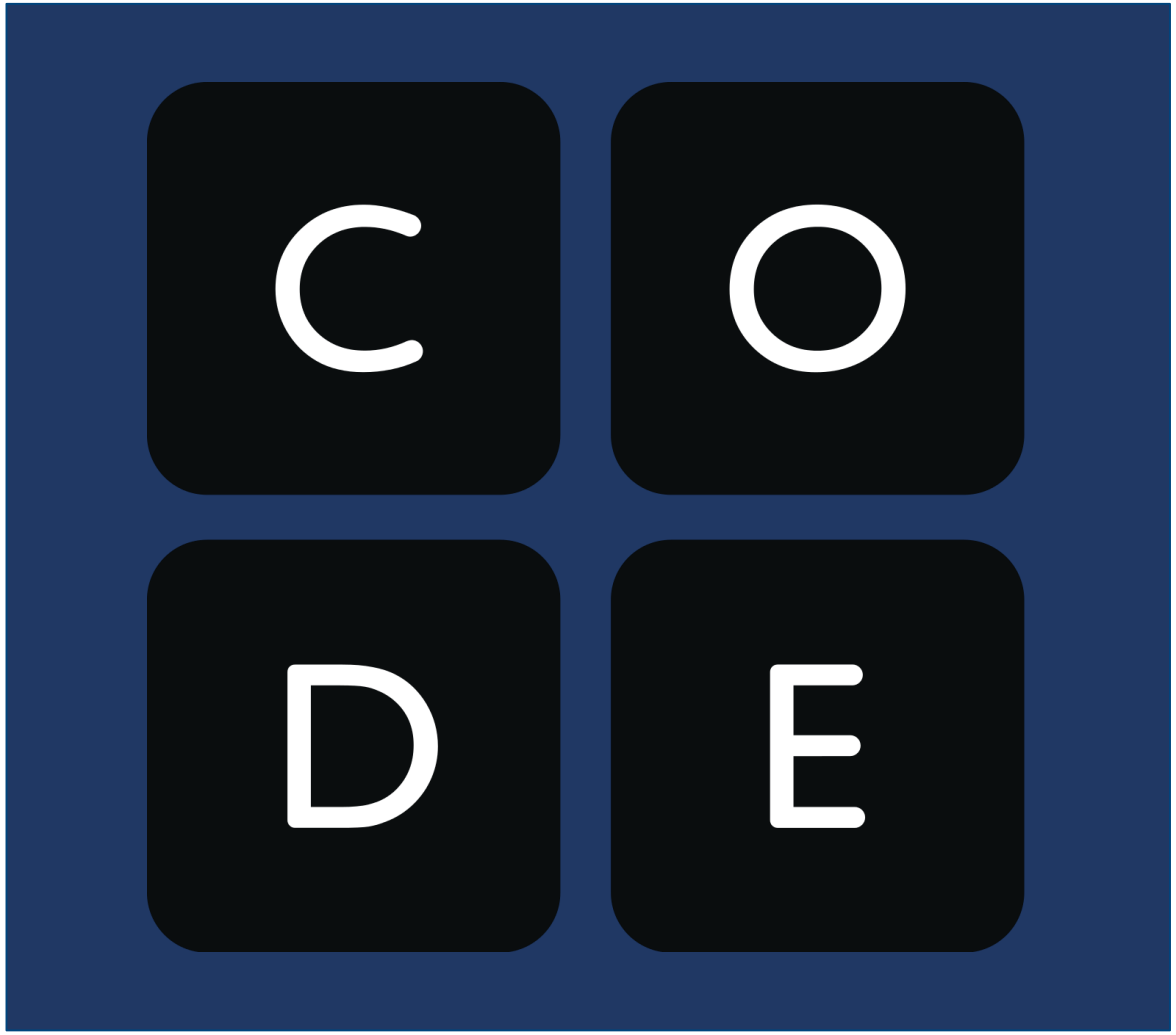
Generators Expression

- Generators assim com as list comprehensions, podem ser geradas automaticamente
- Sua sintaxe é muito semelhante, porém no lugar dos colchetes [] a *Generator Expression* é formada por parênteses ()
- A principal diferença entre uma *list comprehension* e uma *generator expression* é que a primeira irá sempre gerar uma lista com o resultado do processamento enquanto a segunda irá trazer um elemento por vez (*lazy*), consequentemente consumindo menos memória.

generators.py

```
1  numbers = [1, 2, 3, 4, 5, 7]
2  odd_gen_express = (i for i in numbers if i % 2 == 0)
3  for i in odd_gen_express:
4      print(i)
5
```

Challenges Time





Exercicio 1

Considere a seguinte fórmula:

$$\begin{cases} F_1 = 2; \\ F_2 = 1; \\ F_i = 2 * F_{i-1} + G_{i-2} \quad i \geq 3 \end{cases} \qquad \begin{cases} G_1 = 1; \\ G_2 = 2; \\ G_i = G_{i-1} + 3 * F_{i-2} \quad i \geq 3 \end{cases}$$

- Escreva as funções F(n) e G(n) de acordo com a fórmula
- Escreva uma função K(n) que receba um inteiro n > 0 e devolva F(n) e G(n)
  - Para k=2, a função deve devolver os valores 1 e 2
  - Para k=3, a função deve devolver os valores 3 e 8
  - Para k=4, a função deve devolver os valores 8 e 11
- Imprima uma lista de resultados (F(n), G(n)) para K entre 1 e 5, usando *List Comprehension*
- Imprima os mesmos resultador usando *Generator Expression*

Exercicio 2

Na matemática, a Sucessão de Fibonacci (também Sequência de Fibonacci), é uma sequência de números inteiros, começando normalmente por 0 e 1, na qual, cada termo subsequente corresponde à soma dos dois anteriores. A sequência recebeu o nome do matemático italiano Leonardo de Pisa, mais conhecido por Fibonacci , que descreveu, no ano de 1202, o crescimento de uma população de coelhos, a partir desta. (https://pt.wikipedia.org/wiki/Sequ%C3%AAncia_de_Fibonacci)

Em termos matemáticos a sequencia é definida com a seguinte fórmula (infinita):

$$F_n = \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n > 1. \end{cases}$$

Pede-se

- Escreva uma função que calcule um determinado elemento da sequencia de Fibonacci, dado sua posição na sequencia
- Escreva uma expressão lambda que faça a mesma operação
- Aplique as funções acima até a sequencia 20 e imprima na tela a lista de valores da sequencia

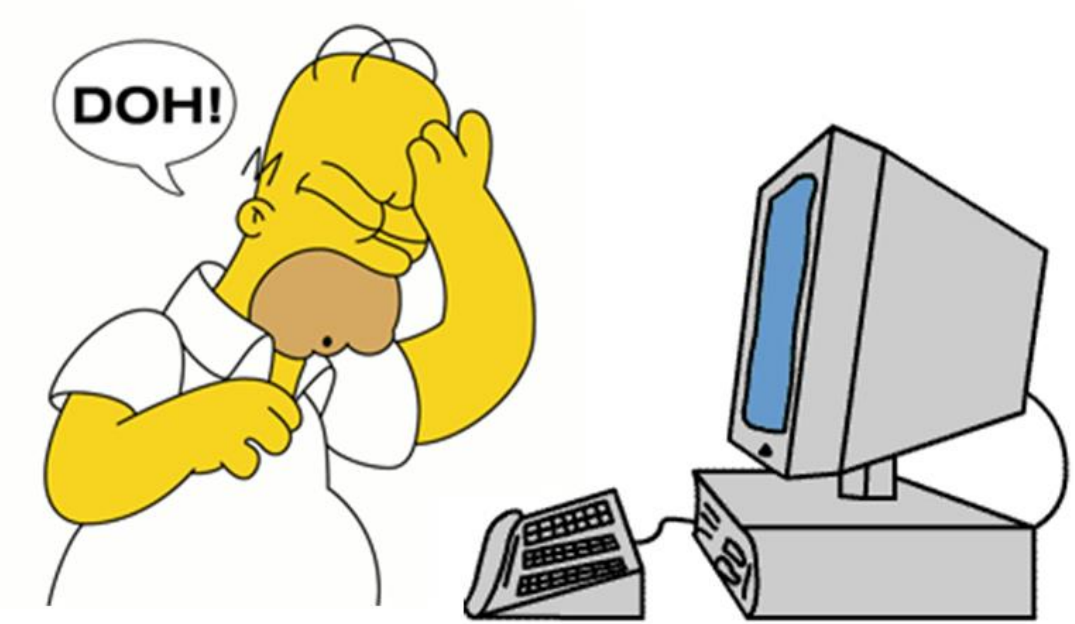
Exercício 3

O teorema de Pitágoras é uma relação matemática entre os comprimentos dos lados de qualquer triângulo retângulo. Na geometria euclidiana, o teorema afirma que: "Em qualquer triângulo retângulo, o quadrado do comprimento da hipotenusa é igual à soma dos quadrados dos comprimentos dos catetos."

$$a^2 = b^2 + c^2 \Rightarrow \quad a = \sqrt{b^2 + c^2}$$
$$b = \sqrt{a^2 - c^2}$$
$$c = \sqrt{a^2 - b^2}$$

- Crie um função que receba 3 valores que representarão os lados de um triângulo. Considere que não importa a ordem que serão fornecidos os valores, podendo ser fornecido primeiro a hipotenusa e depois os catetos, ou primeiro os catetos e depois a hipotenusa, etc.
- Dados os 3 valores, imprima na tela se trata-se de um triângulo-retângulo ou não.

Orientação a Objetos



- Herança
- Polimorfismo
- Abstração
- Encapsulamento
- Associação (Composição / Agregação)
- Generalização / Especialização
- Dependencia
- Classe
- Instância / Objeto

Orientação a Objetos

- O statement *class* indica uma uma definição de uma nova classe
- A função *__init__(self, args...)* define o construtor da classe, sendo o primeiro argumento obrigatoriamente a própria instância da classe (*self*)
- Classes podem ter variáveis de classe que são compartilhadas entre todas as instâncias dessa mesma classe e variáveis de instância que são exclusivas a cada instância
- Instâncias podem receber atributos dinamicamente

oo_example.py

```
class Parent:
    # Class variable
    count = 0

    # constructor
    def __init__(self, name, age):
        # instance variables
        self.name = name
        self.age = age
        Parent.count += 1

    # overriding str
    def __str__(self):
        return 'Name: {}, Age: {}'.format(self.name, self.age)


p1 = Parent('Joao', 10)
p2 = Parent('Maria', 11)
print(p1)
print(p2)
print(Parent.count)
```

Orientação a Objetos

- Herança é definida quando na definição da classe é indicada a classe da qual ela extenderá

oo_example.py

```
16  class Child(Parent):
17
18      def __init__(self, name, age, school_grade):
19          super().__init__(name, age)
20          self.school_grade = school_grade
21
22      def __str__(self):
23          return super().__str__() + ', School Grade: {}'.format(self.school_grade)
24
25  p = Parent('Joao', 50)
26  c = Child('Joaozinho', 10, 5)
27
```

Orientação a Objetos

- Herança multipla funciona da mesma forma, inserindo-se outras classes que deverão ser extendidas

oo_example.py

```
16  class Walker:
17
18      def walk(self):
19          print('Walking...')
20
21      def run(self):
22          print('Running...')
23
24  class Child(Parent, Walker):
25
26      def __init__(self, name, age, school_grade):
27          super().__init__(name, age)
28          self.school_grade = school_grade
29
30      def __str__(self):
31          return super().__str__() + ', School Grade: {}'.format(self.school_grade)
32
```

Orientação a Objetos

- O encapsulamento consiste em "esconder" os atributos de uma classe de forma que não possam ser acessador diretamente
- Em Python não é comum a criação de métodos get/set para atributos caso eles não tenham a real necessidade de serem privados
- A necessidade de privar o acesso a um atributo se dá quando o controle do seu valor é necessário
- Para tornar um atributo privado, basta iniciar seu nome com um duplo underscore *__attrib*
- Os *statements* ***@property*** e ***@attr.setter*** é a forma "Pythonica" de definir métodos *getter* e *setter*

properties.py

```
1   class Account(object):
2
3       def __init__(self, number):
4           number = number
5           self.__special = False
6           self.__balance = 0
7
8       def deposit(self, amout):
9           self.__balance += amout
10
11      def withdrawal(self, amout):
12          if not self.__special and amout > self.__balance:
13              raise Exception('Balance not enough')
14          self.__balance -= amout
15
16      def upgrade(self):
17          self.__special = True
18
19      @property
20      def balance(self):
21          return self.__balance
22
```

Orientação a Objetos

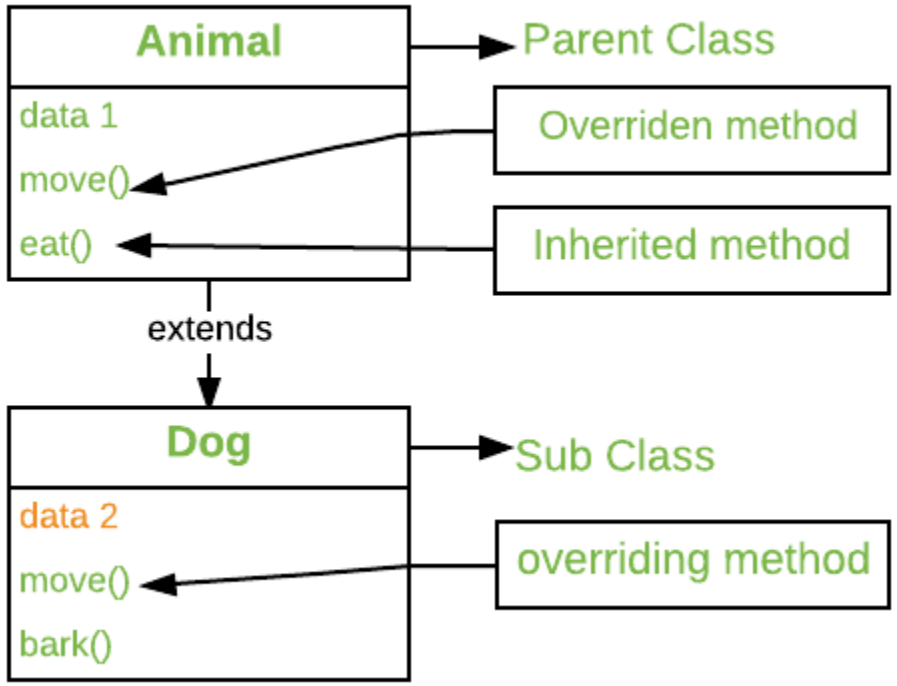
- Em Python não existe o conceito de interfaces para definição de comportamentos, pois tem suporte a herança múltipla
- Para a definição de classes abstratas, Python provê uma infraestrutura denominada ABC (Abstract Base Classes)
- Para tornar uma classe abstrata, basta extender a classe ABC
- O *statement* ***@abstractmethod*** instrui o interpretador de que esse método não está implementado e caso uma nova instancia dessa classe seja criada, um erro de execução será retornado
- O método ***super()*** é utilizado para acesso ao objeto pai

abstract.py

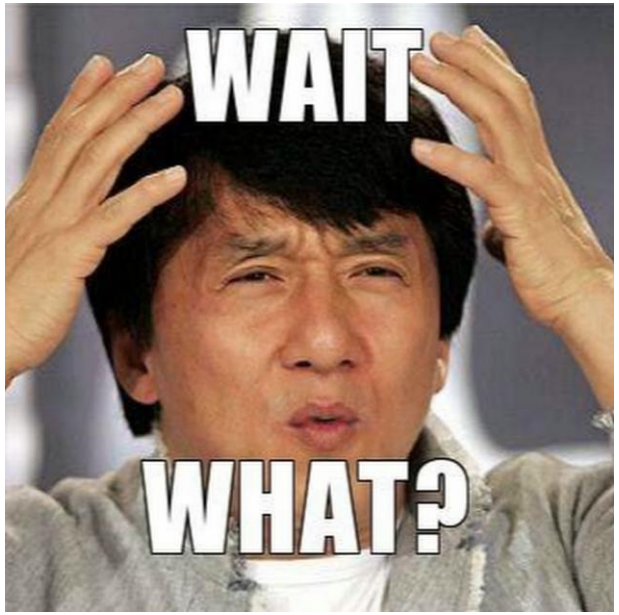
```
1  from abc import ABC, abstractmethod
2
3  class Employee(ABC):
4      def __init__(self, name, salary):
5          self.name = name
6          self.salary = salary
7
8      @abstractmethod
9      def bonus(self):
10         pass
11
12 class Operator(Employee):
13     def bonus(self):
14         return self.salary / 2
15
16 class Manager(Operator):
17     def bonus(self):
18         return super().bonus() + self.salary / 3
19
20 class Director(Manager):
21     def bonus(self):
22         return super().bonus() + self.salary * 2
23
```

Orientação a Objetos

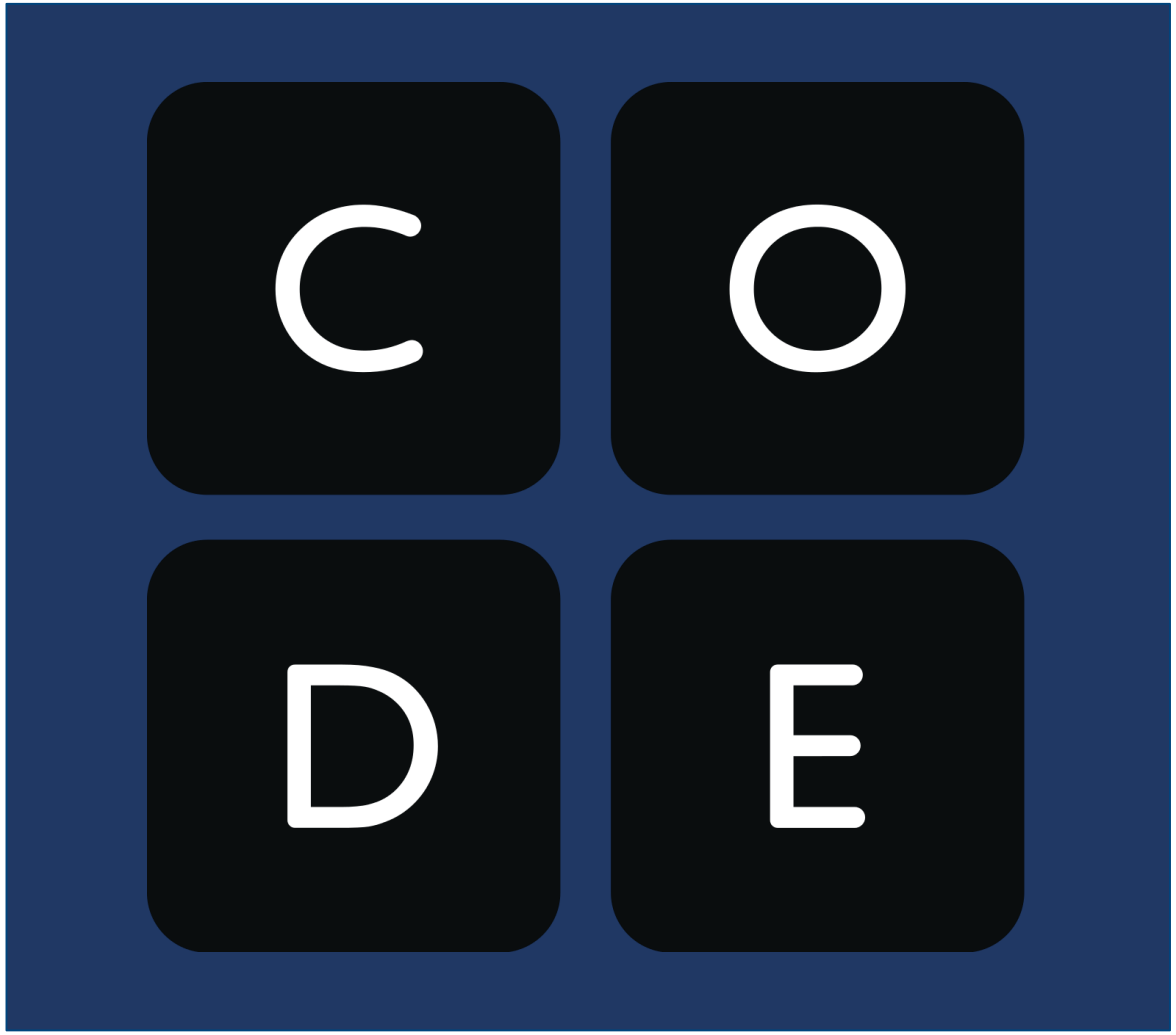
- Sobrescrita de métodos (*overriding*) é permitida, bastanto para isso escrever na subclasse o mesmo nome do método com a mesma quantidade de parâmetros



- Sobrecarga de métodos (*overloading*) não é possível em Python, em virtude da tipagem dinâmica de variáveis, porém é possível simular esse comportamento através de *Default Parameters*



Challenges Time





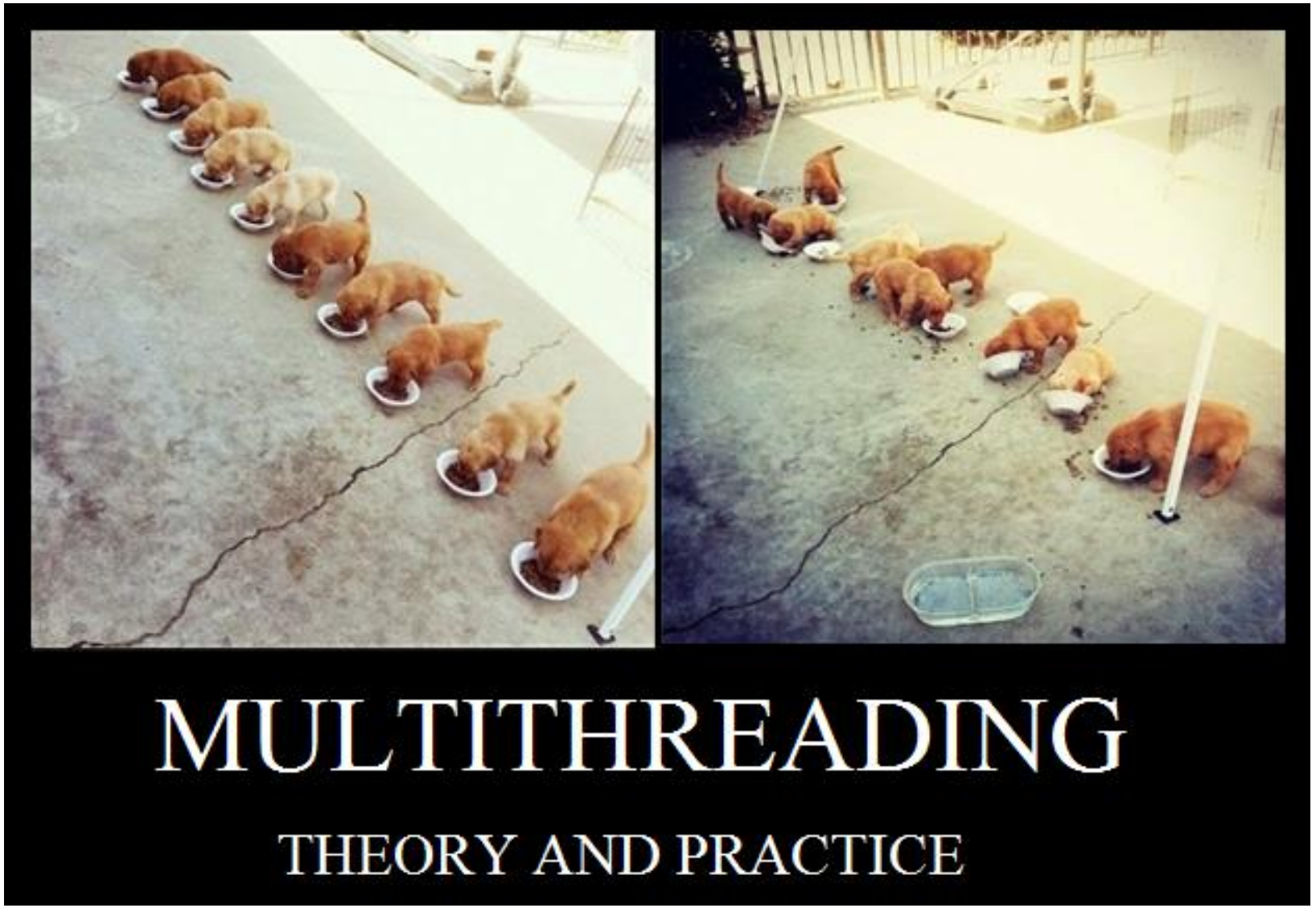
Exercício 4

Dado um arquivo customers.cvs com as seguintes informações sobre clientes:

- id
- name
- gender
- birthday
- email
- active

- Crie a classe *Customer* que represente a estrutura acima
- Efetue a leitura do arquivo, criando uma lista de instâncias de *Customer* (1 instância por linha do arquivo)
- Crie uma lista com todos os Customers ativos (active = True), ordenados por id, salve o conteudo dessa lista em um arquivo do tipo csv (actives.csv)
- Crie uma lista com todos os Customers inativos cujo gênero seja masculino (gender = Male), ordenados por nome. Salve os elementos dessa lista em um arquivo do tipo txt (male_inactives.txt, 1 elemento por linha)

Multithreading



Multithreading

- Pode-se enteder por *Thread* uma linha ou encadeamento de execução de um determinado trecho de código
- É uma eficiente forma de dividir um processo em tarefas que podem ser executadas concorrentemente
- A execução de múltiplas *Threads* simultâneas está diretamente ligada ao hardware e ao sistema operacional
- *Threads* compartilham o mesmo espaço de dados do processo criador, podendo compartilhar informações ou se comunicar com outras *threads* de forma mais simples e eficaz do que entre processos
- Pode-se dizer que threads são *light-weight processes* pois não sobrecarregam o consumo de memória, sendo sua execução mais "*barata*" do que de um processo comum



Multithreading

- Python possui um módulo que fornece suporte a programação multithread denominado *threading*
- Métodos do módulo *threading*
  - threading.activeCount() − retorna o número de threads ativas
  - threading.currentThread() − retorna o número da thread corrente
  - threading.enumerate() − retorna uma lista com todas as threads ativas
- O módulo *threading* também provê a classe *Thread* que provê os métodos
  - run() − método de entrada de uma thread
  - start() − inicia a execução de uma thread
  - join([time]) − aguarda a finalização da thread
  - isAlive() − verifica se a thread ainda está em execução
  - getName() − retorna o nome da thread
  - setName() − atribui um nome à thread

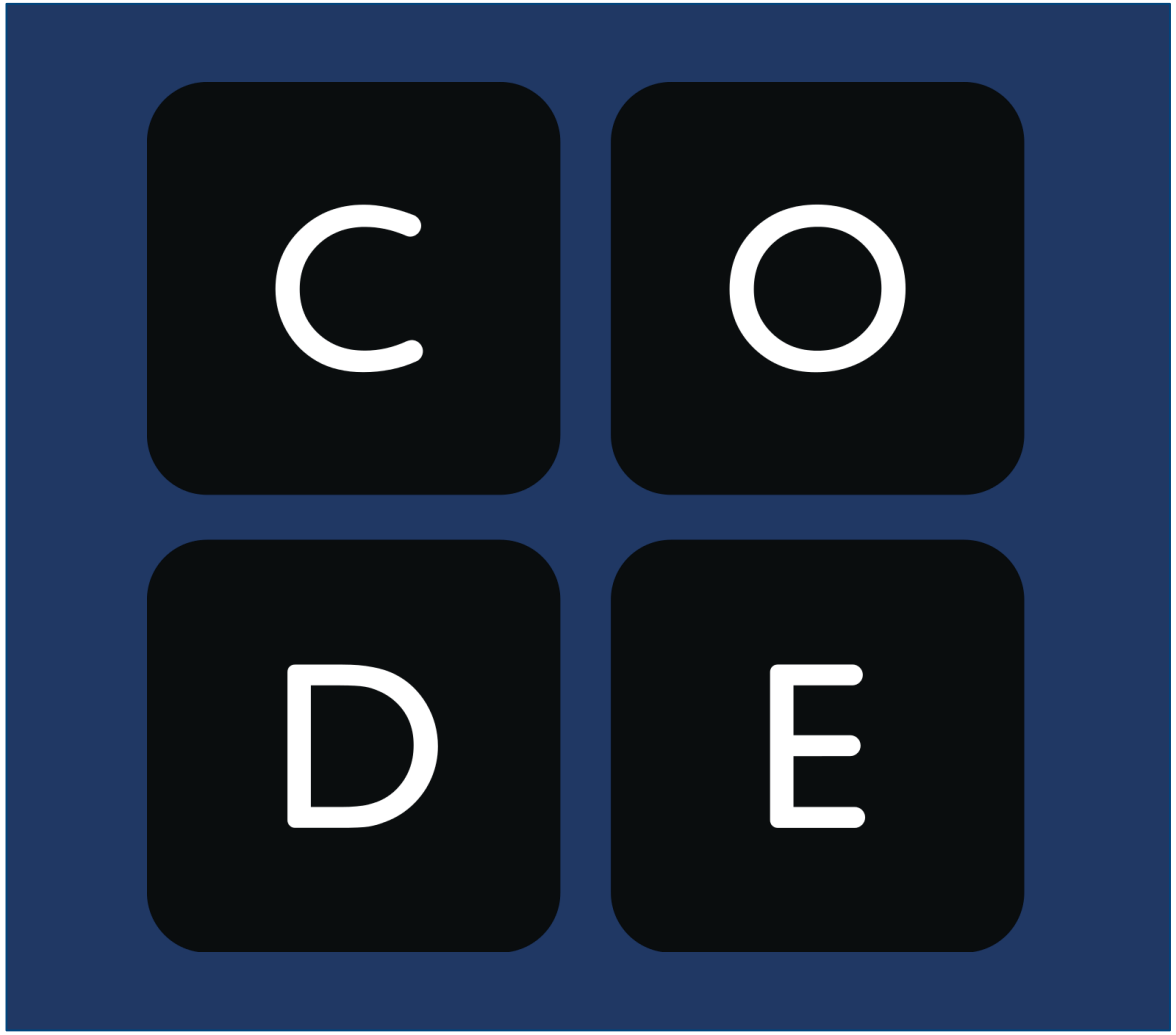
Multithreading

- Para implementar uma *thread* utilizando o *threading module*, basta extender a classe *Thread*
- Sobrescrever o método *__init__* com os parâmetros adicionais da *thread*
- Sobrescrever o método *run* implementando a lógica que a *thread* deverá executar ao iniciar

thread1.py

```
1   import threading, time
2
3   class MyThread(threading.Thread):
4       def __init__(self, times, delay):
5           threading.Thread.__init__(self)
6           self.counter = times
7           self.delay = delay
8
9       def run(self):
10          while self.counter:
11              print("{} - Iterating: {}".format(self.getName(), self.counter))
12              time.sleep(self.delay)
13              self.counter -= 1
```

Challenges Time





Exercício 5

- Escreva um programa que calcule os valores das tabuadas do 1 ao 10 de forma simultânea (1 thread por tabuada)
- Imprima na tela a lista com os resultados de cada tabuada

Referências



- https://www.python.org/
- https://python.org.br/
- http://www.diveintopython.net/
- https://www.w3schools.com/python
- https://www.codecademy.com/tracks/python
- https://www.jetbrains.com/pycharm/


THANK YOU
DANKE
DANK U
TAK
GRACIAS
ĎAKUJEM
ДЗЯКУЙ
ありがとう
ТЕШЕККÜР ЕДЕРИМ
धन्यवाद
TAKK
감사합니다
GO RAIBH MAITH AGAT
Ευχαριστώ
GRAZIE
СПАСИБО
OBRIGADO
شكرا
MERCI
DZIĘKUJĘ
ĎAKUJEM
MULTUMESC
HVALA
KIITOS
СПАСИБІ
תודה
多謝



Facens

AQUI TEM ENGENHARIA